

Sprint Documentation

Sprint 15	Start Date:10/04/2026	Final Date:14/04/2026
Projetc Name:	civitas-frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas – AMS	
Team Members		
Guilherme Henrique de Oliveira		

Sprint Backlog

Task #	Description	Start Date	Final date
160	Contexto Inteligente no Frontend - Suporte para IA	10/04/2026	14/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
160	Criar uma camada de contexto no frontend que forneça informações estruturadas para a IA permitindo respostas mais consistentes, contextualizadas e alinhadas com o estado atual da aplicação (usuário, tela, dados e ações disponíveis).	Guilherme Henrique de Oliveira e	Finish	15	15

Sprint Description

Objetivo

Criar uma camada de contexto no frontend que forneça informações estruturadas para a IA permitindo respostas mais consistentes, contextualizadas e alinhadas com o estado atual da aplicação (usuário, tela, dados e ações disponíveis).

✓ Critérios de Aceite

- Criar um contexto global (ex: React Context ou equivalente) para centralizar informações relevantes para a IA.

Definir quais dados devem compor o contexto (ex: usuário logado, permissões, tela atual, dados exibidos, filtros ativos).

Garantir que o contexto seja atualizado dinamicamente conforme navegação e interações do usuário.

Estruturar os dados de forma padronizada e legível para consumo da IA.

Implementar função/helper para montar payloads de contexto antes de chamadas para IA.

Evitar envio de dados desnecessários ou sensíveis.

Permitir fácil expansão futura do contexto sem quebrar integrações existentes.

Observações

- Priorizar simplicidade e clareza na estrutura do contexto (evitar overengineering).
- Pensar no contexto como uma “memória de curto prazo” da aplicação.
- Evitar acoplamento direto com componentes específicos.
- Considerar performance: não recriar contexto inteiro a cada render.
- Validar o impacto no consumo da API da IA (payload muito grande = custo + latência).
- Preparar estrutura pensando em múltiplos cenários (chat, sugestões automáticas, assistente interno).

Sprint Results

Foi estruturado o contexto técnico do projeto frontend civitas-frontend com base na organização atual do repositório. Foi criada a documentação da codebase na pasta `.planning/codebase/`, incluindo arquitetura, convenções, integrações, stack, estrutura, testes e pontos de atenção do sistema. O material foi adaptado ao frontend, respeitando os padrões já existentes do projeto, sem alterar identidade visual, estilos globais ou componentes sensíveis.

ARCHITECTURE.md

Explica a **arquitetura geral do frontend**.

Mostra como o projeto está organizado em alto nível: App Router do Next.js, layouts, páginas, componentes reutilizáveis, hooks, modelos e fluxo entre interface e API.

CONCERNS.md

Lista os **pontos de atenção e riscos do projeto**.

Serve para registrar problemas, inconsistências, limitações técnicas ou coisas que podem causar manutenção difícil no futuro, como duplicações, ausência de testes ou scripts faltando.

CONVENTIONS.md

Documenta as **convenções do código**.

Mostra os padrões que o projeto já segue e que devem ser respeitados: organização de arquivos, uso de componentes compartilhados, tipagem, padrões de formulários, consumo de API e forma de escrever código sem fugir do estilo existente.

INTEGRATIONS.md

Descreve as **integrações externas e internas** do frontend.

Mostra como o sistema conversa com API, variáveis de ambiente, bibliotecas externas, fontes, ícones e outros serviços realmente usados no projeto.

STACK.md

Resume a **stack tecnológica** do projeto.

Lista frameworks, bibliotecas e ferramentas principais usadas no frontend, como Next.js, React, TypeScript, Tailwind, Axios, Zustand e scripts de execução/build.

STRUCTURE.md

Explica a **estrutura de pastas e arquivos** do repositório.

Ajuda qualquer pessoa a entender onde ficam páginas, componentes, hooks, modelos, docs, utilitários e outros blocos do sistema.

TESTING.md

Mostra o **estado dos testes** no projeto.

Diz se existem ou não testes automatizados, qual é a cobertura atual, o que foi encontrado no repositório e quais validações hoje parecem depender mais de teste manual do que de suíte automatizada.

